

Dynamic verification

Slide 1 · 0–2 min

Dynamic verification

Every technique must discharge a named verification claim.

Design the checking path → Generate and reuse → Read temporal obligations

Teaching script

Open with a decision the audience must make in this area. Ask which result they currently treat as sufficient evidence and keep that answer for the closing discussion.

The lesson selects a practical argument from the book. The study handout preserves the detailed examples, citations and worked artifacts.

Source: ch08:26–74, ch09:83–143, ch11:73–157, ch12:40–62

Learning outcomes

Slide 2 · 2–5 min

Learning outcomes

Demonstrate these decisions on a real plan row.

- Choose a scoreboard identity without losing an ordering check.
- Separate stimulus legality from distribution.
- Read assertion activation, timing and reset.
- Define a reproducible regression and triage policy.

Teaching script

Read the outcomes as observable tasks. Ask learners to identify which one they can already demonstrate and which one currently depends on undocumented judgment.

At closure, require a concrete artifact or explanation for an outcome rather than a general expression of confidence.

Source: ch08:26–74, ch09:83–143, ch11:73–157, ch12:40–62

Route through the lesson

Slide 3 · 5–7 min

Route through the lesson

Four blocks, each ending with a decision.

- Design the checking path
- Generate and reuse
- Read temporal obligations
- Operate the regression

Teaching script

Allow eighteen minutes for each block: a concept, its engineering consequence, a worked interpretation and a short discussion. Keep the discussion tied to the evidence available in the example.

Reserve the final eleven minutes for the assigned exercise and application to the learners' work. The timing is a course design allocation, not an engineering estimate.

Source: ch08:26–74, ch09:83–143, ch11:73–157, ch12:40–62

Separate environment responsibilities

Slide 4 · 7–11 min

Separate environment responsibilities

Observation and measurement must not alter behavior.

- Keep stimulus, driving and monitoring distinct.
- Keep checking independent of coverage collection.

Teaching script

Identify the failure mode of each responsibility rather than treating the taxonomy as a framework convention. A monitor that waits on a slow consumer can change the behavior it was meant to observe.

Use passive observation as the reuse test. The environment must continue to observe correctly when another actor drives the interface.

Source: ch08:26–74, ch08:75–145

The scoreboard key is an engineering decision

Slide 5 · 11–16 min

The scoreboard key is an engineering decision

Reordering tolerance can erase an ordering check.

- Choose a key from observable identity.
- Account for independent concurrency sources.
- Retain explicit ordering obligations.

Teaching script

Contrast an in-order queue with a keyed structure. A content-based lookup may find the right object despite the wrong order, which changes the property being checked.

Ask learners what identifies concurrent DMA work and whether the identity is observable independently of the generator. At end of test, require unmatched expected work to become a failure rather than disappear.

Source: ch08:146–283

A predictor can be wrong

Slide 6 · 16-21 min

A predictor can be wrong

Golden applies to a specification, not to confidence in code.

- Review the reference model's origin.
- Compare at a compatible abstraction boundary.

Teaching script

Distinguish a model that is itself the specification from an independently written transfer function and from an architectural co-simulation reference.

A disagreement between design and predictor requires specification review. Co-simulation also requires aligned architectural observations and delivery of relevant asynchronous events to both models.

Source: ch08:284-351

Discussion: tolerate reordering?

Slide 7 · 21–25 min

Discussion: tolerate reordering?

Name the evidence that would settle the decision.

- A content-keyed scoreboard passes reordered responses.
- The interface requires ordering within a stream.
- What check has been lost?

Teaching script

Expected answer: successful lookup establishes a matching result but does not enforce the required stream order. Keep that ordering obligation explicit in the checker.

Require the proposed key or auxiliary structure to distinguish independent concurrency from illegal reordering within one ordering domain.

Source: ch08:146–283

Legality and distribution differ

Slide 8 · 25–29 min

Legality and distribution differ

A solvable model can still exclude the intended scenario.

- Review the legal set separately from probability.
- Measure the generated distribution.

Teaching script

Explain why changing weights is not the same as changing legal behavior. A satisfiable over-constraint can silently remove the feature under test while every randomization succeeds.

An empty block of coverage cells is a reason to inspect the generator and its constraints before concluding that the corner merely needs more runtime.

Source: ch09:83–143, ch09:144–402

A seed is not a complete reproducer

Slide 9 · 29–34 min

A seed is not a complete reproducer

Execution context belongs in the run record.

- Record source revision, tool build and configuration.
- Preserve random-stability assumptions.

Teaching script

Ask learners to reproduce a run after a tool or environment change using the same seed. Explain why the seed alone does not identify the same experiment.

The chapter's random-stability guidance constrains how new objects, threads and draws are introduced. Record the actual reproducible context rather than relying on a developer's working directory.

Source: ch09:403–446, ch12:63–94

UVM supplies reusable structure

Slide 10 · 34–39 min

UVM supplies reusable structure

It does not supply the plan, checker or coverage intent.

- The interface agent is the reuse boundary.
- A register mirror is not a functional scoreboard.

Teaching script

Explain the active/passive switch using block-to-system reuse. A framework makes this structural change mechanical only if the monitor and checker were designed independently of the local driver.

The register model tracks register behavior described by its source; it does not validate all functional consequences of register programming. Generation from a machine-readable description also leaves that description's correctness to review.

Source: ch10:33–92, ch10:204–293, ch10:314–327

Discussion: the framework is complete

Slide 11 · 39–43 min

Discussion: the framework is complete

Name the evidence that would settle the decision.

- The UVM topology, sequences and register layer run.
- The feature has no pass/fail criterion.
- What is still missing?

Teaching script

Expected answer: verification intent and the checking evidence that would discharge it. Framework completeness does not create a verification plan.

Ask for one concrete feature row and the observation that would falsify it. The answer should identify actual behavior, not a missing framework class.

Source: ch10:314–327

An assertion is a sampled statement

Slide 12 · 43–47 min

An assertion is a sampled statement

Clocking and reset define what is evaluated.

- Identify the sampling event.
- Separate same-tick and future obligations.
- State how reset discards attempts.

Teaching script

Work from a specification sentence to a timing obligation. Concurrent assertions use sampled values, so a waveform interpretation must identify the sampling instant rather than reading continuous time casually.

Explain disable iff as cancellation of in-flight attempts, not merely another implication antecedent. Keep the exact reset behavior in the source property visible during review.

Source: ch11:73–157

Stable can look backward

Slide 13 · 47–52 min

Stable can look backward

A plausible operator can express the wrong time relation.

- A fresh request may legally introduce a new value.
- The hold obligation concerns the following sampled tick.

Teaching script

Use the chapter's backward-stable and forward-obligation pair. A same-tick use of stable compares with the preceding sample, potentially rejecting a valid fresh offer.

Also inspect withdrawal: if the antecedent disappears before the check, the wrong formulation can stop observing exactly when the hold obligation matters. The timing diagram is evidence about the operator's semantics, not a substitute for intent review.

Source: ch11:20–72, ch11:187–423

Cover the antecedent

Slide 14 · 52–57 min

Cover the antecedent

A clean implication can be unexercised.

- Distinguish satisfaction from vacuity.
- Use a witness for the condition that starts the check.

Teaching script

Ask for the trace that activates an implication. A property that never encounters its antecedent can report no failure without evaluating the intended obligation.

The same property text carries different intent under assert, assume and cover. In formal, assumptions remove traces instead of independently demonstrating that the environment satisfies them.

Source: ch11:158–174, ch11:588–617

Discussion: all assertions pass

Slide 15 · 57–61 min

Discussion: all assertions pass

Name the evidence that would settle the decision.

- The traffic never stalls.
- A hold-while-stalled property never fails.
- Which witness should the review demand?

Teaching script

Expected answer: a cover or equivalent witness of the antecedent that starts the hold obligation, together with evidence of the consequent evaluation.

Reject a bare assertion pass count as evidence that stalled behavior was exercised. The aim is to expose inactive checking, not to improve the appearance of the report.

Source: ch11:588–617

Tiers are admission policies

Slide 16 · 61–65 min

Tiers are admission policies

Runtime budgets belong to suite design.

- Place fast static checks before costly runs.
- Give each tier a purpose and entry rule.

Teaching script

Have learners distinguish a fast commit gate from a broad nightly exploration tier. The same test may serve different decisions, but the admission rule and runtime budget must be stated.

A gate too slow for its intended cadence invites bypass. The remedy is a justified tier design, not silently dropping the expensive failure class from verification.

Source: ch12:40–62, ch12:95–110

Triage before deep debugging

Slide 17 · 65–70 min

Triage before deep debugging

Failure signatures must preserve useful distinctions.

- Reduce duplicate failures.
- Separate design, environment and infrastructure causes.
- Keep a reproducible representative.

Teaching script

An over-generic signature merges distinct defects; an over-specific signature turns one defect into many apparent failures. Ask what observations make a signature stable and discriminating.

Measure rediscovery as well as new discovery. The objective is to reduce repeated debug effort while retaining the evidence needed to reproduce each underlying failure.

Source: ch12:111–139

A ranked suite does not explore new behavior

Slide 18 · 70–75 min

A ranked suite does not explore new behavior

Selection needs an independent exploration budget.

- Investigate flakes instead of training teams to ignore them.
- Keep unranked exploration alongside selection.

Teaching script

Explain what a clean rerun of a flaky test does and does not establish. It does not erase the original failing observation; an explicit investigation must identify the relevant uncertainty.

Regression ranking exploits past observations. Preserve a tier that explores beyond that history, especially after design changes or new coverage requirements.

Source: ch12:140–196, ch12:197–321

Discussion: rerun until green

Slide 19 · 75–79 min

Discussion: rerun until green

Name the evidence that would settle the decision.

- An intermittent failure passes on rerun.
- The dashboard replaces the original result.
- What evidence was discarded?

Teaching script

Expected answer: the initial failure and the conditions under which it occurred. Retain the run context and classify the flake; do not redefine success as eventually obtaining a clean sample.

Ask whether the suspected cause is design behavior, environment nondeterminism or infrastructure. The diagnosis determines the investigation, and none is established by a clean rerun alone.

Source: ch12:140–196

Exercise: choose the revealing antecedent

Slide 20 · 79–85 min

Exercise: choose the revealing antecedent

Equivalent formulas can expose different vacuity.

- Compare $ok \text{ implies not err}$ with $err \text{ implies not ok}$.
- Assume err is rare and deliberately constructed.
- Choose a formulation and the necessary activation witness.

Teaching script

Exercise 11.4 (judgement). A review is handed two properties over the same two signals, $ok \rightarrow !err$ and $err \rightarrow !ok$, each with a cover on its antecedent. Name the condition under which each of the two passes vacuously, state the rule §11.7 gives for choosing between two logically equivalent formulations, and apply that rule to select one of the two on the premise that err is the rare, deliberately constructed condition. State also what §11.7 requires in place of a bare antecedent cover when the antecedent is ordinary traffic, and name the cover of §11.6 the chapter offers as the model of the stronger witness.

Assign book Exercise 11.4 in full. The slide summarizes the task; the handout reading must include both vacuity conditions and the stronger witness required for an ordinary-traffic antecedent.

Expected answer: the first passes vacuously when ok is false, the second when err is false. Prefer the deliberately constructed err antecedent and cover it. For an ordinary-traffic antecedent, cover the interesting timing circumstance as well; the chapter uses c_full_bypass , which isolates the full-FIFO bypass condition as that stronger witness.

Source: ch11:588–617, ch11:424–587

What to change on Monday

Slide 21 · 85–87 min

What to change on Monday

Make one existing verification decision reviewable.

- Audit one scoreboard key.
- Reproduce a run from its recorded manifest.
- Find an assertion without an activation witness.

Teaching script

Ask each learner to choose one of these actions in an existing project. Require a small, inspectable deliverable and identify the person who will review it.

Keep the change connected to the lesson's evidence limits. A new dashboard, script or model is useful only if it strengthens the claim the plan row is meant to establish.

Source: ch08:26–74, ch09:83–143, ch11:73–157, ch12:40–62

Scope and next reading

Slide 22 · 87–90 min

Scope and next reading

Use the handout for the detailed artifacts and sources.

- Read the Dynamic verification handout and book Chapters 8–12.
- Detailed UVM APIs and complete environment implementation remain in the handout.

Teaching script

Revisit the outcomes and ask the room to state a limitation of the strongest result discussed today. A complete answer identifies a model boundary, an assumption, a missing oracle or the scope of the underlying evidence.

Assign the detailed reading to the relevant chapters. The omitted topics are explicit scope boundaries, not claims that those topics are unimportant.

Source: ch08:26–74, ch09:83–143, ch11:73–157, ch12:40–62